Set Up Hybrid Proxy Locally in Secondary Instance Group (SIG) Environments

Use the hybrid proxy Vite plugin to run Storefront Next and SFRA/SiteGenesis side by side at one origin during local development when connected to an on-demand sandbox. The steps in this documentation apply primarily to local development. At the end of this setup, you can navigate between Storefront Next and SFRA/SiteGenesis pages at `http://localhost:5173` without visible redirects.

> **Warning**
>
> Hybrid proxy is for local development with a sandbox (ODS) only. It works only with `pnpm dev` and must not run in production.
>
> For Managed Runtime (MRT) and production deployments, use Cloudflare eCDN origin routing between Storefront Next and SFRA/SiteGenesis.

Contents

- Hybrid Proxy Features
- Before You Begin
- Configure the Hybrid Proxy
- Set Routing Rules
- Understanding Request Flow
- Understanding Cookie Behavior
- Troubleshoot Common Issues
- Override Route Matching (Optional)
- See Also

Hybrid Proxy Features

Side-by-Side Local Development

Run Storefront Next and SFRA/SiteGenesis simultaneously at a single origin (`localhost:5173`) with no visible redirects or session loss. Navigate freely between Storefront Next pages and SFRA/SiteGenesis pages as if they were one unified storefront.

expression format (`http.request.uri.path matches`) as your production configuration. Copy your `HYBRID_ROUTING_RULES` value from your eCDN origin rules, or keep it in sync with them, so local development routing matches production.

Configuration Flexibility

No assumptions are made about which pages live on Storefront Next versus SFRA/SiteGenesis. You define the split entirely through `HYBRID_ROUTING_RULES`. Any combination of migrated and legacy pages is valid; include only the routes you've built in Storefront Next.

Feature Flag Safeguard

A dedicated `HYBRID_PROXY_ENABLED` flag controls the proxy. It defaults to `false`, so the proxy doesn't activate unless you explicitly turn it on. The plugin is also guarded by a `mode === 'development'` check in `vite.config.ts`, so the proxy doesn't run in a production build.

Before You Begin

Make sure that:

- You run your storefront with `pnpm dev`.
- You have access to an B2C Commerce sandbox that runs SFRA/SiteGenesis.
- You have the required storefront environment variables.
- You complete "Configure Business Manager" in Set Up Your Storefront with Hybrid Auth. Those Business Manager settings are also required when using the hybrid proxy.

Configure the Hybrid Proxy

Step 1: Configure the Application (All Environments)

These settings must be present in your `.env` for local development, and in your MRT environment variables for staging and production.

```
cp .env.default .env
```

2. Enable hybrid mode.

```
PUBLIC__app__hybrid__enabled=true
```

3. Set `PUBLIC__app__hybrid__legacyRoutes` to a JSON array of routes that belong to SFRA/SiteGenesis. When a user clicks a `<Link>` to one of these paths, the client-side navigation middleware forces a full-page load so the CDN routes the request to SFRA/SiteGenesis.

```
PUBLIC__app__hybrid__legacyRoutes
```

OR

Supports exact paths and React Router-style parameterized patterns:

```
PUBLIC__app__hybrid__legacyRoutes
```

> **Note**
> This list is the inverse of `HYBRID_ROUTING_RULES`. If a route isn't in your routing rules (SFRA/SiteGenesis owns it) and a Storefront Next page links to it, add it here. If it's missing, React Router tries to render the route client-side and shows a 404.

Step 2: Configure Proxy Plugin (Local Development Only)

These settings apply only to the Vite dev server. They have no effect in production builds.

4. Set `HYBRID_PROXY_ENABLED=true`.

```
HYBRID_PROXY_ENABLED=true
```

5. Set `SFCC_ORIGIN` to the full HTTPS URL of your B2C Commerce sandbox.

which paths stay on Storefront Next.

```
HYBRID_ROUTING_RULES='(http.reque
```

7. Set `PUBLIC__app__defaultSiteId` so that the proxy can build SFRA/SiteGenesis-prefixed paths. The proxy fails fast at startup if this variable is missing when `HYBRID_PROXY_ENABLED=true`.

```
PUBLIC__app__defaultSiteId=RefArc
```

8. Optional: Set `HYBRID_PROXY_LOCALE` if your SFRA/SiteGenesis locale path differs from your storefront fallback locale.

```
HYBRID_PROXY_LOCALE=en-GB
```

9. Start the local development server with `pnpm dev`.

The proxy is disabled by default in `.env.default` and only activates when `HYBRID_PROXY_ENABLED=true`.

Set Routing Rules

Use `HYBRID_ROUTING_RULES` to define route ownership. To compare route split patterns, see Choose a Hybrid Routing Matrix.

- Paths that match your expression go to Storefront Next.
- Paths that don't match go to B2C Commerce (SFRA/SiteGenesis) via the proxy.
- Define any split between migrated and legacy pages. Include only the routes you've built in Storefront Next.

The rule syntax matches Cloudflare eCDN origin rules:

```
http.request.uri.path matches "<regex>
```

Combine clauses with `or` and wrap the expression in parentheses:

Required Route Patterns

Include these patterns so React Router server endpoints continue to work:

Pattern	Required for
`^/resource.*`	React Router resource routes
`^/action/.*`	React Router actions
`.*\.data.*`	Production parity in shared rule sets

Page route patterns (optional)

Add patterns for Storefront Next pages that your team has already migrated:

Pattern	Route
`^/$`	Homepage
`^/login.*`	Login page
`^/logout.*`	Logout
`^/signup.*`	Registration
`^/reset-password.*`	Password reset
`^/account.*`	Account pages
`^/product.*`	Product detail pages
`^/category.*`	Category and PLP pages
`^/search.*`	Search results
`^/social-callback.*`	Social login callback

Full Rule Set Example

This example keeps homepage, auth, product, category, search, account, and server endpoints on Storefront Next:

Paths Excluded From Proxying

The proxy doesn't forward these paths to B2C Commerce:

- `/@*`, `/__*` (Vite internals)
- `/src/*`, `/node_modules/*` (Vite-served source files)
- `*.data` (React Router data requests)
- `/mobify/*` (SCAPI proxy paths handled by React Router)
- Static assets (`.js`, `.css`, `.png`, `.woff2`, and similar extensions)

The proxy always forwards SFRA/SiteGenesis static assets under `/on/demandware.static/*` and `/on/demandware.store/*` to B2C Commerce.

Understanding Request Flow

The hybrid proxy processes each development request in this order:

1. Vite receives the request at `localhost:5173`.
2. The hybrid proxy middleware evaluates the request path.
3. If the path matches `HYBRID_ROUTING_RULES`, React Router handles the request in Storefront Next.
4. If the path doesn't match, the proxy forwards the request to `SFCC_ORIGIN`.
5. For proxied storefront paths, the proxy rewrites to SFRA/SiteGenesis format:
    - `/cart` -> `/s/{siteId}/{locale}/cart`
6. The proxy rewrites response cookies and response body links for localhost continuity.

Understanding Cookie Behavior

Hybrid auth requires that both Storefront Next and SFRA/SiteGenesis share the same session cookies (`dwsid`, `cc-*`). The proxy keeps them in sync in three ways:

other cookie attributes (`Secure` , `SameSite` , `HttpOnly`). Localhost is a secure context, so `Secure` cookies work on `http://localhost` .

2. **Storefront Next server cookies (Layer 2)**: Storefront Next sets its own session cookies (`dwsid` , access token, refresh token, etc.) server-side via the auth middleware. Storefront Next writes them directly to `localhost` –they require no rewriting. The proxy doesn't modify them.
3. **Client-side cookie interception (Layer 3)**– localhost workaround only: The proxy injects an inline script at the top of every proxied HTML page. The script patches `document.cookie` to apply the same `Domain=localhost` rewrite to any cookies set by SFRA/SiteGenesis's own JavaScript. SFRA/SiteGenesis's client-side scripts check `window.location.protocol` to decide whether to include `Secure` on cookies–on `http://localhost` they omit it, producing cookies the browser silently rejects. This layer exists solely to compensate for localhost's non-HTTPS context and has no equivalent in the eCDN-based hybrid implementation used in production.

Troubleshoot Common Issues

Client-side navigation to SFRA/SiteGenesis routes shows a 404

If a `<Link>` in Storefront Next navigates to an SFRA/SiteGenesis route and React Router shows a 404 or error boundary instead of the SFRA/SiteGenesis page, the route is probably missing from `PUBLIC__app__hybrid__legacyRoutes` .

Add the missing path to the array:

```
1  PUBLIC__app__hybrid__legacyRoutes='["/
```

Set this variable in every environment–local development and production (MRT).

your Cloudflare eCDN origin rules. If they diverge, local behavior doesn't match production routing.

Missing routing rules

If a Storefront Next path is missing from `HYBRID_ROUTING_RULES`, the request can fall through to SFRA/SiteGenesis and trigger SFRA/SiteGenesis redirects. The proxy logs a warning for SFRA/SiteGenesis 404 redirect patterns, for example:

```
1 [Hybrid Proxy] SFCC returned a redirec
2   This usually means your HYBRID_ROUTI
```

Add the missing route pattern to `HYBRID_ROUTING_RULES`.

Invalid locale in SFRA/SiteGenesis path rewriting

SFRA/SiteGenesis expects `/s/{siteId}/{locale}/path`. If `HYBRID_PROXY_LOCALE` doesn't match your SFRA/SiteGenesis locale configuration, requests can return 404 responses or redirects.

Unsupported compression

The proxy rewrites response bodies after decompressing `gzip`, `brotli`, and `deflate`. If B2C Commerce returns another compression format, the proxy skips body URL rewriting.

Production-mode guard

The plugin runs only in development mode through the `mode === 'development'` guard in `vite.config.ts`.

Override Route Matching (Optional)

Warning

We strongly recommend using the default `shouldRouteToNext` matcher. Overriding it means your local dev routing no longer mirrors the Cloudflare eCDN expression format

The default matcher (`shouldRouteToNext`) parses Cloudflare expression syntax. Use a custom matcher only when you intentionally diverge from production-style route expressions.

```
// vite.config.ts
import { hybridProxyPlugin, shouldRout

hybridProxyPlugin({
  routeMatcher: (pathname, rules) => {
    if (pathname === "/my-custom-page"
    if (pathname === "/legacy-only") r
    return shouldRouteToNext(pathname,
  },
});
```

To replace the matcher entirely, use:

```
routeMatcher: (pathname) => myOwnRouti
```

The callback receives `pathname` and `routingRules`. Return `true` to route to Storefront Next, or `false` to proxy to B2C Commerce.

If the matcher throws an error, the proxy fails safe and passes the request to React Router.

See Also

- Use Storefront Next in a Hybrid Implementation
- Project Configuration
- B2C Commerce API: CDN APIs for Hybrid Implementations
- B2C Commerce Architecture

Try for free

MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

Component Library
APIs
Trailhead
Sample Apps
AppExchange

Events and Calend
Partner Communi
Blog
Salesforce Admins
Salesforce Archite